Clase 8: ¿Cómo Aprende una IA? Pesos, Neuronas y Error

Repaso de la Clase Anterior ¿Qué se aprendió la clase pasada?

- **Pandas:** Manipulación de DataFrames, selección, filtrado, agrupación.
- **Seaborn:** Visualización estadística avanzada (histogramas, barras, dispersión, boxplots, mapas de calor).
- **Preparación de datos para ML:** Manejo de valores nulos, codificación de variables categóricas con One-Hot Encoding, creación de columnas derivadas, exportación de dataset limpio.

Conexión con la clase de hoy:

- Ya se tienen los datos limpios y listos para Machine Learning.
- Pero **¿qué hace un modelo de ML con esos datos?** ¿Cómo "aprende"?
- Hoy se responderá esa pregunta desde cero, entendiendo los conceptos de **pesos**, **bias**, **neurona artificial** y **error**.

Agenda de la Clase

1. **Explicación Teórica**:
    - ¿Qué significa que una IA "aprenda"?
    - El modelo más simple: la **neurona artificial** (perceptrón).
    - Conceptos de **peso** (w), **bias** (b) y **entrada** (x).
    - Función de activación: la **función escalón**.
    - Cálculo del **error** y su importancia en el aprendizaje.
    - Introducción intuitiva al **descenso del gradiente**.
2. **Actividad Práctica en Clase**:
    - Implementar una neurona artificial en Python puro (sin librerías).
    - Simular el aprendizaje de una **compuerta lógica AND** ajustando pesos y bias manualmente.
    - Visualizar cómo el error disminuye al ajustar los parámetros.
3. **Actividad Independiente** (5 min):
    - Implementar una neurona para la compuerta **OR** y **NOT**.

4.Explicación Teórica: Pandas, Seaborn y Preparación de Datos

4.1. ¿Qué significa que una IA "aprenda"?

En la IA clásica, un programador escribía **reglas explícitas**: "si pasa esto, haz aquello". En la IA moderna, no se escriben reglas: se le dan **ejemplos** a la máquina y ella **descubre las reglas por sí sola**.

Analogía: Imagine que quiere enseñarle a un niño a distinguir entre perros y gatos. No le da una lista de reglas ("si tiene orejas puntiagudas, es gato..."). Le muestra muchas fotos y le dice "esto es un perro", "esto es un gato". El niño, con el tiempo, **aprende los patrones** por sí solo.

Eso es exactamente lo que hace una IA: **aprende patrones a partir de ejemplos**.

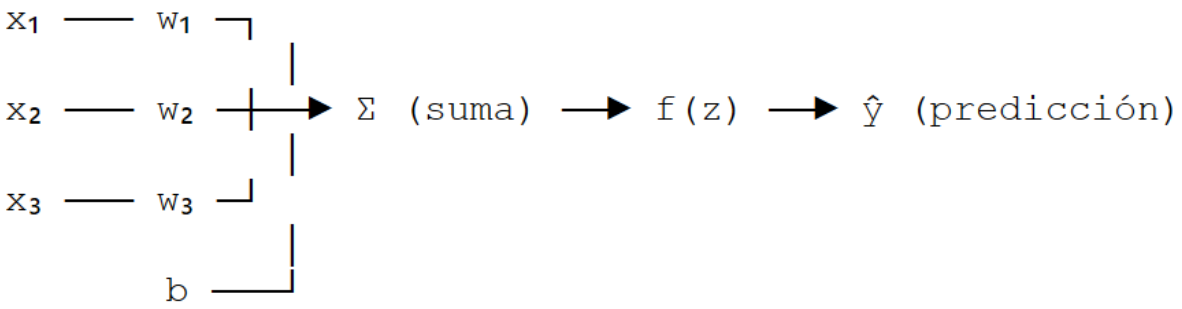
En términos técnicos:

- **Aprender = ajustar parámetros internos** (pesos y bias) para que las predicciones se acerquen a las respuestas correctas.
- **Los parámetros** son números que la máquina modifica en cada intento.
- **El error** es la diferencia entre lo que predijo y lo que debía predecir. El objetivo es **minimizar ese error**.

4.2. La Neurona Artificial (Perceptrón)

El modelo más simple de IA es la **neurona artificial**, inspirada en las neuronas biológicas. Fue propuesta por Frank Rosenblatt en 1958 y se llama **perceptrón**.

Estructura de una neurona artificial:



Donde:

- x_1, x_2, x_3: Entradas (features, datos).
- w_1, w_2, w_3: Pesos (weights). Indican la importancia de cada entrada.
- b: Bias (sesgo). Es un valor adicional que permite ajustar la predicción.
- Σ: Suma ponderada. Se calcula como $z = w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3 + b$.
- f(z): Función de activación. Decide la salida final.
- ŷ: Predicción (salida).

En palabras simples:

1. Se multiplica cada entrada por su peso.
2. Se suman todos esos productos y se le agrega el bias.
3. Se pasa esa suma por una función de activación.
4. Se obtiene la predicción.

4.3. Pesos (w) y Bias (b)

- **Los pesos (**w**)** determinan cuánta importancia tiene cada entrada en la decisión final.
    - Si un peso es muy alto, esa entrada influye mucho.
    - Si un peso es cercano a 0, esa entrada casi no influye.
    - Si un peso es negativo, la entrada **resta** en lugar de sumar.
- **El bias (**b**)** permite que la neurona active su salida incluso cuando todas las entradas son 0.
    - Sin bias, la neurona siempre necesitaría que alguna entrada sea distinta de 0 para activarse.
    - El bias es como un "umbral" que se desplaza.

Analogía de la vida real: puede imaginar que está decidiendo si salir a correr. Sus entradas podrían ser:

- x_1: ¿Hace buen clima? (1 = sí, 0 = no)
- x_2: ¿Tengo tiempo libre? (1 = sí, 0 = no)
- x_3: ¿Me siento con energía? (1 = sí, 0 = no)

Los pesos reflejan cuánto le importa cada factor:

- $w_1 = 0.3$ (el clima importa, pero no es lo principal)
- $w_2 = 0.5$ (el tiempo es importante)
- $w_3 = 0.8$ (la energía es lo más importante)

El bias sería su "estado de ánimo general":

- $b = -0.5$ (usted es un poco flojo por naturaleza)

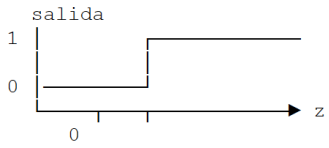
Con esto, la neurona calcula: $z = 0.3 \cdot x_1 + 0.5 \cdot x_2 + 0.8 \cdot x_3 - 0.5$
Si $z > 0$, sale a correr. Si no, se queda en casa.

4.4. Función de Activación: la Función Escalón

La función de activación decide si la neurona "se activa" o no. La más simple es la **función escalón** (step function):

```
def escalon(z):
  if z >= 0:
    return 1
  else:
    return 0
```

Gráficamente:



- Si $z >= 0$, la salida es 1.
- Si $z < 0$, la salida es 0.

Ventaja: Es muy simple y fácil de entender.
Desventaja: No es derivable, lo que complica el uso del descenso del gradiente (por eso en clases posteriores veremos funciones más avanzadas como la sigmoide).

4.5. El Error y su Importancia en el Aprendizaje

El error es la diferencia entre la predicción de la neurona (ŷ) y el valor real esperado (y).

```
error = y_real - y_predicho
```

El objetivo del aprendizaje es ajustar w y b para que el error sea cada vez menor.

Ejemplo simple:

Si se quiere que la neurona aprenda la compuerta AND:

- Entrada (0, 0) → se espera 0
- Entrada (0, 1) → se espera 0
- Entrada (1, 0) → se espera 0
- Entrada (1, 1) → se espera 1

Si la neurona predice 1 cuando debería predecir 0, el error es -1. Si predice 0 cuando debería predecir 1, el error es +1.

¿Cómo se corrige? Ajustando los pesos y el bias en la dirección que reduce el error.

4.6. Introducción al Descenso del Gradiente

El **descenso del gradiente** es el algoritmo que permite ajustar los pesos y bias para minimizar el error. La idea es intuitiva:

Analogía: puede imaginar que está en una montaña con niebla y quiere llegar al punto más bajo. No puede ver el fondo, pero puede sentir la pendiente bajo sus pies. Entonces, da un paso en la dirección que baja más pronunciadamente. Repite esto hasta que ya no puede bajar más.

En IA:

- La "montaña" es la **función de error**.
- Los "pasos" son los ajustes de w y b.
- La "pendiente" es el **gradiente** (derivada del error respecto a cada parámetro).
- El "tamaño del paso" se llama **tasa de aprendizaje** (learning rate, α).

Fórmula simplificada:

```
w_nuevo = w_viejo + α · error · x
b_nuevo = b_viejo + α · error
```

Donde:

- α es un número pequeño (ej. 0.1) que controla qué tan rápido aprende.
- error = y_real - y_predicho.

En esta clase se verá el concepto, y en la siguiente se implementará el descenso del gradiente completo.

Configuración del Entorno (Docker, WSL, VSCode)

5.1. Abrir WSL

Desde PowerShell de Windows:

```
wsl
```

5.2. Navegar al proyecto

```
cd ~/IA/clase8
```

Nota: Asegúrese de tener una carpeta clase8. Si no la tiene, puede crearla:

```
mkdir -p ~/IA/clase8/src
cd ~/IA/clase8
```

5.3. Levantar el contenedor

docker-compose up -d

Si no tiene docker-compose.yml **en** clase8, puede copiarlo de la clase anterior:

```
cp ~/IA/clase7/docker-compose.yml ~/IA/clase8/
cp ~/IA/clase7/Dockerfile ~/IA/clase8/
cp ~/IA/clase7/requirements.txt ~/IA/clase8/
```

5.4. Entrar al contenedor

docker exec -it python_ia bash

5.5. Verificar el entorno

```
which python
# Salida esperada: /opt/venv/bin/python

python -c "import numpy, pandas, matplotlib, seaborn; print('Entorno listo')"
```

Nota: Para esta clase **no se necesitan librerías especiales** porque se implementará la neurona en Python puro. NumPy sí es útil para operaciones vectorizadas, pero no es obligatorio.

6. Actividad en Clase (Guía Paso a Paso)

Objetivo: Implementar una neurona artificial en Python puro y simular su aprendizaje para la compuerta lógica AND.

6.1. Compuerta Lógica AND

La compuerta AND devuelve 1 solo si **ambas entradas son 1**.

Entrada x_1	Entrada x_2	Salida esperada (y)
0	0	0
0	1	0
1	0	0
1	1	1

6.2. Estructura del Script

Crear el archivo neurona_and.py en /app/src/.

```
"""
neurona_and.py
Implementación de una neurona artificial simple para aprender la compuerta AND.
No se usan librerías externas, solo Python puro.
"""
# ============================================================
# 1. Definir la función de activación (escalón)
# ============================================================
def escalon(z):
    """Función de activación escalón."""
    return 1 if z >= 0 else 0

# ============================================================
# 2. Definir la neurona
# ============================================================
def neurona(x1, x2, w1, w2, b):
    """Calcula la salida de la neurona."""
    z = w1 * x1 + w2 * x2 + b
    return escalon(z)

# ============================================================
# 3. Datos de entrenamiento (compuerta AND)
# ============================================================
datos = [
    (0, 0, 0),  # (x1, x2, y_real)
    (0, 1, 0),
    (1, 0, 0),
    (1, 1, 1),
]
```

```
# ============================================================
# 4. Parámetros iniciales (los vamos a ajustar)
# ============================================================
w1 = 0.5
w2 = 0.5
b = -0.7

tasa_aprendizaje = 0.1

# ============================================================
# 5. Entrenamiento manual (varias épocas)
# ============================================================
print("=" * 60)
print("ENTRENAMIENTO DE UNA NEURONA PARA LA COMPUERTA AND")
print("=" * 60)
print(f"Parámetros iniciales: w1={w1}, w2={w2}, b={b}\n")

for epoca in range(10):
  error_total = 0
  print(f"--- Época {epoca + 1} ---")
  for x1, x2, y_real in datos:
    # Predicción
    y_pred = neurona(x1, x2, w1, w2, b)

    # Cálculo del error
    error = y_real - y_pred
    error_total += abs(error)

    # Ajuste de pesos y bias
    w1 += tasa_aprendizaje * error * x1
    w2 += tasa_aprendizaje * error * x2
    b += tasa_aprendizaje * error

    print(f"  Entrada: ({x1}, {x2}) | Real: {y_real} | Pred: {y_pred} | Error: {error}")

  print(f"  Error total: {error_total}")
  print(f"  Parámetros actuales: w1={w1:.3f}, w2={w2:.3f}, b={b:.3f}\n")

  if error_total == 0:
    print(f"La neurona aprendió en {epoca + 1} épocas.\n")
    break

# ============================================================
# 6. Prueba final
# ============================================================
print("=" * 60)
print("PRUEBA FINAL")
print("=" * 60)
for x1, x2, y_real in datos:
  y_pred = neurona(x1, x2, w1, w2, b)
  estado = "OK" if y_pred == y_real else "X"
  print(f"  {estado} Entrada: ({x1}, {x2}) | Esperado: {y_real} | Predicho: {y_pred}")
```

6.3. Ejecutar el Script

Dentro del contenedor, en /app/src/:

```
python neurona_and.py
```

6.4. Salida Esperada (Aproximada)

```
============================================================
ENTRENAMIENTO DE UNA NEURONA PARA LA COMPUERTA AND
============================================================
Parámetros iniciales: w1=0.5, w2=0.5, b=-0.7

--- Época 1 ---
  Entrada: (0, 0) | Real: 0 | Pred: 0 | Error: 0
  Entrada: (0, 1) | Real: 0 | Pred: 0 | Error: 0
  Entrada: (1, 0) | Real: 0 | Pred: 0 | Error: 0
  Entrada: (1, 1) | Real: 1 | Pred: 1 | Error: 0
  Error total: 0
  Parámetros actuales: w1=0.500, w2=0.500, b=-0.700

La neurona aprendió en 1 épocas.

============================================================
PRUEBA FINAL
============================================================
  Entrada: (0, 0) | Esperado: 0 | Predicho: 0
  Entrada: (0, 1) | Esperado: 0 | Predicho: 0
  Entrada: (1, 0) | Esperado: 0 | Predicho: 0
  Entrada: (1, 1) | Esperado: 1 | Predicho: 1
```

Nota: Con los parámetros iniciales ya elegidos, la neurona aprende en la primera época. **Se recomienda modificar los valores iniciales** para que observen cómo el error disminuye a lo largo de las épocas. Por ejemplo:

```
w1 = 0.1
w2 = 0.1
b = 0.0
```

Con esos valores, el aprendizaje será más lento y podrán ver la evolución.

6.5. Ejercicio Guiado: Visualizar el Error

Modificar el script para guardar el error total por época y graficarlo con Matplotlib.

```
import matplotlib.pyplot as plt

# Lista para guardar los errores por época
errores_por_epoca = []

# Dentro del bucle de épocas, después del bucle interno:
errores_por_epoca.append(error_total)

# Al final del script, graficar
plt.figure(figsize=(8, 5))
plt.plot(range(1, len(errores_por_epoca) + 1), errores_por_epoca, marker='o', color='red')
plt.title('Evolución del Error durante el Entrenamiento')
plt.xlabel('Época')
plt.ylabel('Error Total')
plt.grid(True)
plt.savefig('evolucion_error.png', dpi=150)
plt.close()
print("Gráfico guardado: evolucion_error.png")
```

6.6. Preguntas de Reflexión (para discutir en clase)

1. **¿Qué pasa si se inicializa** w1 = 0, w2 = 0, b = 0**?** ¿La neurona aprende? ¿Por qué?
2. **¿Qué pasa si la tasa de aprendizaje es muy alta? (ej.** 1.0**)** ¿La neurona aprende o se "desestabiliza"?
3. **¿Qué pasa si la tasa de aprendizaje es muy baja? (ej.** 0.001**)** ¿Cuántas épocas necesita?
4. **¿Por qué el bias es necesario?** ¿Qué pasaría si b = 0 siempre?
5. **¿Cómo cambiaría el código si se quisiera aprender la compuerta OR en lugar de AND?**

7. Actividad Independiente (Complementaria)

Objetivo: Implementar neuronas para otras compuertas lógicas y reflexionar sobre el aprendizaje.

Instrucciones detalladas

1. **Crear un archivo** neuronas_logicas.py con las siguientes implementaciones:
    - **Compuerta OR:** Devuelve 1 si al menos una entrada es 1.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	1

    - **Compuerta NOT: Devuelve 1 si la entrada es 0, y viceversa (solo una entrada).**

x_1	y
0	1
1	0

2. Para cada compuerta:
    - Definir los datos de entrenamiento.
    - Elegir parámetros iniciales (w1, w2, b).
    - Entrenar la neurona por varias épocas.
    - Mostrar la evolución del error.
    - Verificar que la neurona aprende correctamente.
3. **Responder en el** README.md **del proyecto** las siguientes preguntas:
    - ¿Cuántas épocas necesitó cada compuerta para aprender?
    - ¿Fue necesario ajustar la tasa de aprendizaje? ¿Por qué?
    - ¿Qué diferencias encontró entre la compuerta AND y la OR?
    - ¿Por qué la compuerta XOR **no puede** ser aprendida por una sola neurona? (Pista: investigue sobre el problema de la compuerta XOR y el perceptrón simple).
4. **Subir los cambios a GitHub** con un commit descriptivo.

Fecha de entrega: Antes de la próxima sesión.

8. Cierre

Resumen de la clase

- **Una IA aprende** ajustando sus parámetros internos (pesos y bias) a partir de ejemplos.
- **La neurona artificial (perceptrón)** es el modelo más simple de IA.
- **El error** es la diferencia entre la predicción y el valor real. Minimizarlo es el objetivo del aprendizaje.
- **El descenso del gradiente** es el algoritmo que permite ajustar los parámetros en la dirección correcta.
- **La función escalón** es la función de activación más simple, pero tiene limitaciones.

Conexión con el Proyecto

Aunque sus proyectos aún no tienen un modelo de IA, ya deben entender **cómo funcionan por dentro**. Esto les permitirá, en las próximas clases, no solo usar librerías como Scikit-Learn, sino **comprender qué está pasando** cuando entrenan un modelo.

Próxima clase (Clase 7)

- **Implementación completa del perceptrón** con descenso del gradiente en Python puro.
- **Entrenamiento** con un dataset real y visualización de la frontera de decisión.
- **Comparación** con la implementación de la clase de hoy.

Anexo: Cheat Sheet de la Neurona Artificial

Concepto	Fórmula / Código	Descripción
Suma ponderada	z = w1*x1 + w2*x2 + b	Combinación lineal de entradas y pesos.
Función escalón	1 if z >= 0 else 0	Activa o desactiva la salida.
Predicción	y_pred = escalon(z)	Salida final de la neurona.
Error	error = y_real - y_pred	Diferencia entre lo esperado y lo predicho.
Ajuste de peso	w += α * error * x	Actualiza el peso en dirección que reduce el error.
Ajuste de bias	b += α * error	Actualiza el bias.
Tasa de aprendizaje	α (ej. 0.1)	Controla el tamaño del paso en el ajuste.